AeryxIQ

DRONE PACS AND AERIAL INSPECTION



INSTALLATION

Installation Guide

Requirements, the three deployment paths, certificates, offline installation, upgrades and what to do when something will not start.

Version 1.0.1

On-Premises Edition

November 2026

Amhage Technology Group LLC · aeryxiq.com · michael.romansky@amhage.com

AeryxIQ is a trademark of Amhage Technology Group LLC.

What you are installing

Four containers, one network, four volumes. Nothing else, and nothing outside the machine.

Container	Does
caddy	Terminates HTTPS and passes everything inward. The only container with a port on the host
frontend	Serves the application and proxies its API calls to the backend
backend	The application: imports, grading, reports, evidence hashing
postgres	PostgreSQL 17. Assets, missions, findings, users

A fifth, **nodeodm**, is defined but off. It is the photogrammetry engine that turns photographs into elevation models and orthomosaics, and it is heavy enough to deserve a deliberate decision.

Volume	Holds
`aeryxiq_onprem_postgres_data`	The database
`aeryxiq_onprem_media_data`	Every frame ever imported
`aeryxiq_onprem_caddy_data`	The certificate authority and its certificates
`aeryxiq_onprem_caddy_config`	Caddy's own state

The names are fixed rather than derived from the folder you unzipped into, so that a backup script keeps working after somebody moves the folder.

What leaves the machine

By default, nothing. There is no licence server, no activation, no usage reporting and no telemetry. The installation never contacts Amhage.

Outbound connections happen only where you configure them yourself: an SMTP server if you want invitation emails sent rather than shown on screen, and a public elevation service if you want the "pull elevation from the map" button to work. Both are blank out of the box. The one unavoidable connection is the install itself, which downloads the images from `ghcr.io`; the offline procedure later in this guide removes even that.

Sizing

	Minimum	Comfortable

CPU	4 cores	8 cores
Memory	8 GB	16 GB
Disk	60 GB plus imagery	Imagery, doubled

Imagery is what grows. A 20 megapixel camera on a routine structure survey produces roughly 8 to 12 MB a frame, and a thorough flight over one building runs to several hundred frames. Reckon on your card capacity plus about half again for derived imagery, and watch the figure that `health` reports rather than guessing.

The photogrammetry engine changes this picture entirely. If you intend to generate elevation models, budget 16 GB of memory and several times the disk, and give it its own machine if the reading room is busy.

Installing on Windows

1. Docker Desktop

Install it from `docker.com`, start it, and wait for the whale icon in the system tray to stop animating. On Windows it needs WSL 2, which its own installer will set up and which usually wants a restart.

In Docker Desktop, under Settings, Resources, give it at least 4 CPUs and 8 GB of memory. The defaults on a 16 GB laptop are often lower than that and the first large import is where you find out.

2. Unzip

Unzip `aeryxiq-onprem-1.0.1.zip` somewhere permanent. Not the desktop, and not a folder inside `Downloads` that gets cleared. `C:\AeryxIQ` is a reasonable choice. The folder holds the configuration and the scripts; the data lives in Docker volumes and does not move with it.

3. Start

```
cd C:\AeryxIQ\aeryxiq-1.0.1
powershell -ExecutionPolicy Bypass -File .\up.ps1
```

Add `-WithDemoData` to load the sample content on this first run. The script checks Docker is running, writes `.env` with a freshly generated database password and token signing key, downloads the images, starts the stack, and waits until the application answers before it tells you it is ready.

> **KEEP .ENV**
> It holds the database password and the key that signs every session. Losing the signing key signs everybody out. Losing the database password makes the database unreadable. It is in every backup this product takes, for exactly that reason.

4. Open it and create an administrator

```
https://aeryxiq.localhost

powershell -ExecutionPolicy Bypass -File .\new-admin.ps1
```

5. Start it with Windows, if this is a server

Docker Desktop has a "Start Docker Desktop when you sign in" setting, and the containers are set to `restart: unless-stopped`, so they come back by themselves. That depends on somebody signing in. On a machine that should serve the building whether or not anyone has logged on, install on Linux or use the appliance instead; a Windows server running Docker Desktop behind a locked screen is a support call waiting to happen.

Installing on Linux

1. Docker Engine

```
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER
```

Log out and back in, so the group membership takes effect, then check:

```
docker info
```

2. Unzip and start

```
unzip aeryxiq-onprem-1.0.1.zip
cd aeryxiq-1.0.1
chmod +x *.sh
./up.sh
```

`chmod` is there because a zip file does not always carry the executable bit across platforms.

3. Start it at boot

The containers restart themselves with the Docker daemon, and the daemon starts at boot. If you want the stack brought up explicitly, for instance to control ordering against other services, a unit file does it:

```
[Unit]
Description=AeryxIQ
Requires=docker.service
After=docker.service

[Service]
Type=oneshot
RemainAfterExit=yes
WorkingDirectory=/opt/aeryxiq/aeryxiq-1.0.1
ExecStart=/usr/bin/docker compose up -d
ExecStop=/usr/bin/docker compose down

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl enable --now aeryxiq
```

Reaching it from other machines

Out of the box the address is `aeryxiq.localhost`, which resolves to the loopback address in every browser and therefore works only on the machine itself. To serve the building, set `APP_HOST` in `.env` to the server's name or address and restart:

```
APP_HOST=aeryxiq.example.internal
```

```
./down.sh
./up.sh
```

Leave `tls internal` in the `Caddyfile`. The certificate stays self-issued, which is what a closed site wants; each machine that will use it runs the trust step below once, or accepts the warning. Point a DNS record or a hosts-file entry at the server, and make sure the firewall allows 443 from the subnets that should reach it.

The virtual appliance

A whole machine, configured, that boots into AeryxIQ.

`AeryxIQ-Appliance-1.0.0.ova` is an Ubuntu 24.04 LTS virtual machine with Docker and this bundle already in it. Import it into VMware, VirtualBox, Proxmox or anything else that reads OVF, and it starts the stack on first boot.

Give it	
CPU	4 virtual cores
Memory	8 GB
Disk	60 GB, expand before first boot if you have imagery to put in
Network	Bridged, so it gets an address on your network

First boot

The console prints the address it came up on. Sign in to the console as `aeryxiq` with password `aeryxiq` ; it forces a change immediately, and will not let you keep the old one.

Then open the printed address in a browser and create your administrator over SSH or at the console:

```
cd /opt/aeryxiq
./new-admin.sh
```

THREE THINGS TO DO BEFORE IT IS A SERVER

Change the console password, which it insists on anyway. Put your own SSH key on it and turn off password authentication. Expand the disk before you import imagery, because growing a virtual disk after the fact is a tedious afternoon.

Growing the disk later

Enlarge the virtual disk in the hypervisor, then inside the appliance:

```
sudo growpart /dev/sda 1
sudo resize2fs /dev/sda1
df -h /
```

The certificate

AeryxIQ issues its own HTTPS certificate from a certificate authority created inside the installation on first start. This is a deliberate choice. A publicly trusted certificate has to be requested from the internet, and a product that cannot be installed in a building without a connection is not an on-premises product.

The consequence is the warning on the first visit. The connection is encrypted either way; the browser is telling you it has not met this authority before, which is true.

Removing the warning

Windows, in an administrator PowerShell window:

```
powershell -ExecutionPolicy Bypass -File .\trust-certificate.ps1
```

Linux:

```
sudo ./trust-certificate.sh
```

Both copy the authority's public certificate into the machine's trust store. No private key leaves the container, and the authority can sign for this installation only. Close every browser window afterwards, because browsers cache the old verdict.

Firefox and Chrome keep their own certificate stores on Linux. If the warning persists there, import the same certificate through the browser's own settings.

Rolling it out to a fleet

Extract the certificate once and push it with whatever you already use for machine configuration:

```
docker compose cp caddy:/data/caddy/pki/authorities/local/root.crt aeryxiq-root.crt
```

On Windows that is a Group Policy computer certificate under Trusted Root Certification Authorities. On Linux it is the file dropped into `/usr/local/share/ca-certificates/` followed by `update-ca-certificates`.

Using your own certificate instead

If you have a certificate for an internal name from your own authority, replace the `tls internal` line in the `Caddyfile` with the paths to your certificate and key, mount them into the caddy container, and restart. The rest of the stack is unaffected.

Installing with no internet access

The images are the only thing that needs a connection, and they can travel on a disk.

On a machine that does have access, with Docker installed:

```
docker pull ghcr.io/amhage-technology-group/aeryxiq-backend:1.0.1
docker pull ghcr.io/amhage-technology-group/aeryxiq-frontend:1.0.1
docker pull postgres:17
docker pull caddy:2-alpine

docker save -o aeryxiq-images-1.0.1.tar \
  ghcr.io/amhage-technology-group/aeryxiq-backend:1.0.1 \
  ghcr.io/amhage-technology-group/aeryxiq-frontend:1.0.1 \
  postgres:17 caddy:2-alpine
```

That file is around 1.5 GB. Carry it and the bundle zip to the target machine, then:

```
docker load -i aeryxiq-images-1.0.1.tar
cd aeryxiq-1.0.1
docker compose up -d
```

Use `docker compose up -d` directly rather than `up.sh`, because the script tries to pull first and will stop when it cannot reach the registry. You will need to create `.env` yourself: copy `env.example` to `.env` and fill in the two secrets with long random strings. On the connected machine:

```
python3 -c "import secrets; print(secrets.token_urlsafe(48))"
```

Check the images arrived intact before trusting the installation:

```
docker images | grep aeryxiq
```

> **UPGRADES TRAVEL THE SAME WAY**
>
> Pull the new tags on the connected machine, save, carry, load, change `AERYXIQ_VERSION` in `.env`, and restart.

Optional integrations

All four are blank out of the box, and blank is a working state rather than a broken one.

Email

With no SMTP configured, invitations and password resets are shown on screen for the administrator to pass on. For a single site that is often what you want, and it is one less system to keep credentials for. To send them instead, set the `SMTP_` values in `.env` and restart. Any provider works.

Elevation

`DEM_SERVICE_URL` drives the "pull elevation from the map" button. It needs an outbound connection to a public elevation service. The United States Geological Survey's 3D Elevation Program service is free and needs no key:

```
DEM_SERVICE_URL=https://elevation.nationalmap.gov/arcgis/rest/services/3DEPElevation/ImageServer
```

Left blank, the button is absent rather than broken, which is the right behaviour for a disconnected site.

Photogrammetry

```
NODEODM_URL=http://nodeodm:3000
```

```
docker compose --profile terrain up -d
```

Expect the first job to be slow and memory-hungry. Watch it with `docker stats` before you promise anybody a turnaround time.

Dictation transcription

With `TRANSCRIBE_API_KEY` unset, dictation uses the browser's own live transcript, which never leaves the machine. Setting it sends audio to whichever endpoint you configure. On a site that chose on-premises software for a reason, that is a decision to make deliberately rather than by filling in a blank.

Flight splitting

```
FLIGHT_GAP_MINUTES=45
```

The gap in capture times that starts a new flight during import. Raise it if one sortie is arriving as two studies; lower it if two sorties are arriving as one.

Upgrading

Back up first. Not as advice: as the step.

```
powershell -ExecutionPolicy Bypass -File .\backup.ps1 -To D:\Backups
./backup.sh /backups/aeryxiq
```

Then set the new version in `.env` and restart:

```
AERYXIQ_VERSION=1.0.1
```

```
docker compose pull
docker compose up -d
```

Schema changes are applied at startup. Watch them land rather than assuming:

```
docker compose logs -f backend
```

Then check the version the application reports matches what you asked for:

```
powershell -ExecutionPolicy Bypass -File .\health.ps1
```

Going back

Set `AERYXIQ_VERSION` to the old value and restart. That works when the newer version made no schema change. When it did, restore the backup instead: a database that has been migrated forward is not guaranteed to satisfy the older code, and finding out which way round you are in is not a thing to do at speed.

Uninstalling

```
docker compose down
```

Containers gone, every flight and image kept. To remove the data as well, deliberately and irreversibly:

```
docker compose down -v
```

Then delete the folder. Nothing is written anywhere else on the machine, apart from the certificate if you ran the trust step, which is removed through `certlm.msc` on Windows or by deleting the file from the trust store on Linux.

When it will not start

Docker is not running

On Windows and macOS, `up` starts Docker Desktop itself and waits up to five minutes for its engine, printing what Docker says while it waits. If it still fails, open Docker Desktop: the status in its bottom-left corner has to read *Engine running*, and it may be waiting for you to accept terms or finish installing WSL. If the message says access is denied, add your Windows account to the `docker-users` group and sign out and back in. On Linux, `sudo systemctl start docker`, and if `docker info` fails for your user rather than for the machine, you are not in the `docker` group yet.

Port 80 or 443 is already in use

`up` handles this on its own: if something else holds 443 or 80 (IIS, Skype, another web server), it moves to 8443 and 8080, or the next free alternative, and records the choice in `.env`. To choose the ports yourself, set them in `.env` before starting:

```
HTTPS_PORT=8443
HTTP_PORT=8080
```

The address becomes `https://aeryxiq.localhost:8443`.

The images will not download

The machine needs outbound HTTPS to `ghcr.io` for the install. Behind a proxy, configure it in Docker Desktop's settings or in the daemon's configuration on Linux. If that is not going to happen, the offline procedure earlier in this guide avoids it entirely.

The application never reports healthy

```
docker compose logs backend
```

A refusal to start on a weak signing key is deliberate and the message says so. That happens when `.env` was hand-edited and `JWT_SECRET` ended up short or empty. Generate a long one and restart; everybody signs in again once, and nothing in the database is affected.

The browser cannot find aeryxiq.localhost

Every current browser resolves `*.localhost` to the loopback address without any configuration. An older one, or an unusual corporate setup, may not. Use `https://127.0.0.1` to confirm the stack is up, and add a hosts-file entry if you want the name.

An upload fails on a big folder

One upload is capped at 2 GB. Split the folder; flight splitting works across uploads, so the studies still come out right.

It was working and now it is slow

```
powershell -ExecutionPolicy Bypass -File .\health.ps1
./health.sh
```

That reports what is running and how much disk the volumes have taken. A full disk presents as general slowness long before it presents as an error.

Getting help

Have the output of `health`, the version from the sign-in page, and the last fifty lines of `docker compose logs backend` ready. Those three answer most of the first round of questions.

`michael.romansky@amhage.com` → `aeryxiq.com/services/`